AP HOGESCHOOL ANTWERPEN

Web Frameworks

Syllabus 2026-2027

Kristof Michiels
AP Hogeschool Antwerpen
versie van 21 september 2026

Inleiding

Dit is de gloednieuwe syllabus voor het vak Web Frameworks. Ze vergezelt de slides die bij elke theorieles worden voorzien en de labo's waarin je elke week de leerstof inoefent.

In zo'n tekst kruipt veel werk. Telkens een nieuw hoofdstuk klaar is wordt hier een nieuwe versie van de syllabus gepost. Er wordt naar gestreefd dat de tekst tegen het einde van het semester is uitgegroeid tot een volwaardig handboek dat in staat is jou tot op het niveau van een beginnende junior Angular developer te brengen.

Bij de ontwikkeling van deze syllabus werd generatieve AI gebruikt als ondersteunend hulpmiddel voor onder meer redactie, structurering en uitwerking. Alle inhoud werd door de auteur gecontroleerd, bewerkt en gevalideerd. De auteur draagt de volledige inhoudelijke verantwoordelijkheid.

Inhoud

Hoofdstuk 1 - Het web als applicatieplatform

In een notendop

Wat je na dit hoofdstuk kan

1.1 Waarom het vak Web Frameworks?

1.2 Van document naar applicatie

1.3 Wat een framework is (en wat niet)

1.4 Geen magie: een reactief systeem in twintig regels

1.5 Het spectrum: van strooisel tot platform

1.6 De grote drie

1.7 Angular vandaag - en waarom dat woord "vandaag" ertoe doet

1.8 TypeScript: types als contract

1.9 Het Node-ecosysteem: de werkplaats

1.10 Je gereedschap: de Angular CLI en je eerste applicatie

1.11 De volledige stack: waar dit vak naartoe werkt

Onder de motorkap: wat ng serve echt doet

Veelgemaakte fouten

Met AI aan je zij: gedateerd Angular herkennen

Oefeningen

Verder lezen

Hoofdstuk 1 - Het web als applicatieplatform

In een notendop

Het web begon als een systeem van documenten en is een platform voor toepassingen geworden. Dit hoofdstuk legt uit hoe dat kwam, waarom we een toepassing vandaag opsplitsen in een frontend en een backend, en welk probleem daarbij ontstaat: het scherm gelijk houden met de data. Een framework lost dat probleem op door de rollen om te keren: het framework roept jouw code aan, niet omgekeerd. Je bouwt zelf een reactief systeem in twintig regels, je situeert de frameworks van vandaag, en je zet met de Angular CLI je eerste toepassing op.

Wat je na dit hoofdstuk kan

Aan het einde van dit hoofdstuk draait er een Angular-toepassing op je laptop. Ze doet nog niet heel veel; dat komt vanaf hoofdstuk 2. Maar je hebt haar zelf gegenereerd, opgestart, beperkt aangepast, en je weet wat elk bestand erin komt doen. Belangrijker: je kan uitleggen **waarom** ze bestaat. Waarom bouwt vrijwel elke organisatie haar webapplicaties vandaag met een framework, terwijl jij vorig jaar in Web Technology en Web Programming technisch hebt bewezen dat het ook zonder kan? Wat is een **Single-Page Application**, en wat is er "single" aan? Waarom splitsen we een applicatie in een frontend en een backend, en wat levert dat op? Waarom bouwt zowat de hele frameworkwereld op TypeScript? En wat gebeurt er tussen het intypen van `ng serve` en een werkende pagina in de browser?

Dit opleidingsonderdeel heet Web Frameworks, in meervoud. Je leert in dit hoofdstuk de veelheid aan frameworks situeren: van vanilla JavaScript over lightweight tools als Alpine.js en HTMX tot volledige platformen als Angular. In het eerste labo bouw je zelf het bewijs dat frameworks geen technische mirakels zijn: we maken een werkend reactief systeem in twintig regels vanilla JavaScript. In het tweede labo leer je de Angular CLI kennen, het gereedschap waarmee je de rest van het semester werkt.

Dit is het enige hoofdstuk van de syllabus waarin je vooral leest en het bouwen in dienst ervan staat. Dat is een bewuste keuze. De volgende zes hoofdstukken bevatten veel meer code, en elke regel daarvan is beter te begrijpen als het model uit dit hoofdstuk goed in je hoofd zit.

1.1 Waarom het vak Web Frameworks?

In de vakken Web Technology en Web Programming leerden we je handmatig HTML, CSS en JavaScript schrijven. Ook nu blijven we je dat vragen, ook voor Angular. Waarom, als AI dat in seconden doet? Code produceren is goedkoop geworden. Code begrijpen, beoordelen en corrigeren is dat niet. Het model produceert, jij beoordeelt - en beoordelen kan alleen wie de regels zelf kent.

Eerst de plaats van dit vak, want die verklaart de aanpak. Web Frameworks is het sluitstuk van de webvakken. In Web Technology leerde je het platform kennen: HTML, CSS, de DOM. In Web Programming kwamen de taal en de server erbij: JavaScript en TypeScript, Node en npm, een Express-server met MySQL erachter, de Fetch API ertussen. Dit vak voegt daar weinig losse onderwerpen aan toe. Het voegt alles samen tot wat de industrie er dagelijks mee bouwt: volwaardige toepassingen, van scherm tot database. Bijna elk begrip dat je dit semester tegenkomt, staat dus op een fundament dat je al hebt, en deze syllabus zegt er telkens bij welk. Nieuw is vooral de schaal, en het gereedschap dat die schaal beheersbaar maakt.

Er zijn twee redenen waarom je dat gereedschap grondig moet kennen.

De eerste: frameworks zijn de taal waarin professionele webontwikkeling vandaag gebeurt, en dus ook de taal waarin AI-modellen het grootste deel van hun frontend-code genereren. Wie in een bedrijf aan een webapplicatie werkt, werkt vrijwel zeker in Angular, React of Vue. Wie een model vraagt om "een dashboard" of "een boekingsformulier", krijgt framework-code terug. Die code kan je alleen beoordelen als je het framework kent.

De tweede is specifieker, en je komt ze dit semester voortdurend tegen: **AI-modellen genereren massaal verouderde Angular-code**. Angular is tussen 2023 en 2026 grondig veranderd, en het internet - de trainingsdata van elk model - staat vol code van daarvoor. Een model dat jij vandaag om Angular-code vraagt, geeft je geregeld patronen die in jouw versie niet meer bestaan, niet meer werken of als bad practice gelden. In dit vak leer je Angular zoals het vandaag is. Het kader "Met AI aan je zij" aan het einde van elk hoofdstuk leert je die oude code herkennen. Zien wat er niet klopt is een kernvaardigheid van een moderne (web) developer.

En dus geldt hier dezelfde afspraak als vorig jaar, om dezelfde reden: **je schrijft je code zelf.** Niet omdat AI verboden terrein is, maar omdat handmatig kunnen bouwen het bewijs is dat het model in je hoofd zit. En omdat de vaardigheidstoets op het einde van dit OLOD een toets zonder AI is. Wie de labo's uitbesteedt aan een chatbot, voelt zich twaalf weken productief maar faalt vrijwel zeker op het examen zelf. Dat overkomt elk jaar studenten. Zorg dat jij er niet bij bent 🙏 .

Hoe we in dit vak met AI werken

Elke oefening in deze syllabus en elk labo draagt één van drie labels:

- **zonder AI**: je gebruikt geen AI. Dit label staat op de vaardigheidstoets en op oefeningen die daar rechtstreeks op voorbereiden
- **zonder AI aanbevolen**: je mag AI gebruiken. Dit label staat op de meeste drills

- **met AI**: je gebruikt AI en dat is het punt van de oefening. In dit vak betekent dat meestal: AI-output beoordelen en moderniseren. Wat we oefenen, zijn jouw analytische skills.
-

Daarnaast heeft elk labo een kort blok "AI in dit labo" dat zegt wat in die fase van het vak wel en niet verstandig is, telkens met de reden erbij. De rode draad is overal dezelfde: AI mag je helpen *begrijpen* (leg dit concept uit, verklaar deze foutmelding), niet *produceren* (schrijf deze component voor mij). De proef op de som: elke regel die je indient, moet je mondeling kunnen uitleggen.

1.2 Van document naar applicatie

Om te begrijpen waarom frameworks bestaan, moet je eerst weten welk probleem ze eigenlijk oplossen. Dat probleem is ontstaan op het moment dat het web iets anders werd dan waarvoor het oorspronkelijk ontworpen was.

Het web van documenten

Het web is ontworpen als een systeem van documenten. Een browser (de client) vraagt een HTML-bestand op, een server stuurt het terug, de browser toont het. Elke klik op een link begint de conversatie opnieuw, voor een nieuw document. Dat model ken je uit Web Technology, en voor alles wat op een statisch document lijkt - een artikel, een portfolio, een nieuwssite - is het vandaag nog altijd het juiste model.

Maar veel van wat we "websites" noemen, zijn geen documenten meer. Gmail, Google Docs, Spotify, je online bank, het leerplatform van onze hogeschool: dat zijn *applicaties* die in een browser draaien. Ze onthouden dingen (welke mail is geselecteerd, wat zit er in je winkelmandje), ze reageren voortdurend op de gebruiker, en ze herladen niet de hele pagina bij elke actie. De browser is voor die programma's geen documentviewer maar een besturingssysteem: het platform waarop de applicatie draait.

Waar wordt de pagina gemaakt? Frontend en backend

Bij elke webapplicatie kan je één vraag stellen die alles verklaart: **waar wordt de HTML gemaakt?**

In het klassieke antwoord gebeurt dat op de server. De gebruiker klikt, de server haalt gegevens uit de database, giet ze in een HTML-sjabloon en stuurt een volledige pagina terug. Zo werkt PHP, zo werkt een Express-server met templates, of Flask voor de Python-kenners, zo werkt het grootste deel van het web tot vandaag. De hele applicatie leeft op één plek, en de browser toont alleen het resultaat.

In het moderne antwoord gebeurt het in de browser. Dat kan door twee dingen die je al kent: JavaScript kan de DOM herschrijven zonder de pagina te herladen, en JavaScript kan met `fetch` zelf gegevens ophalen bij een server. Combineer die twee en je krijgt een **Single-Page Application**

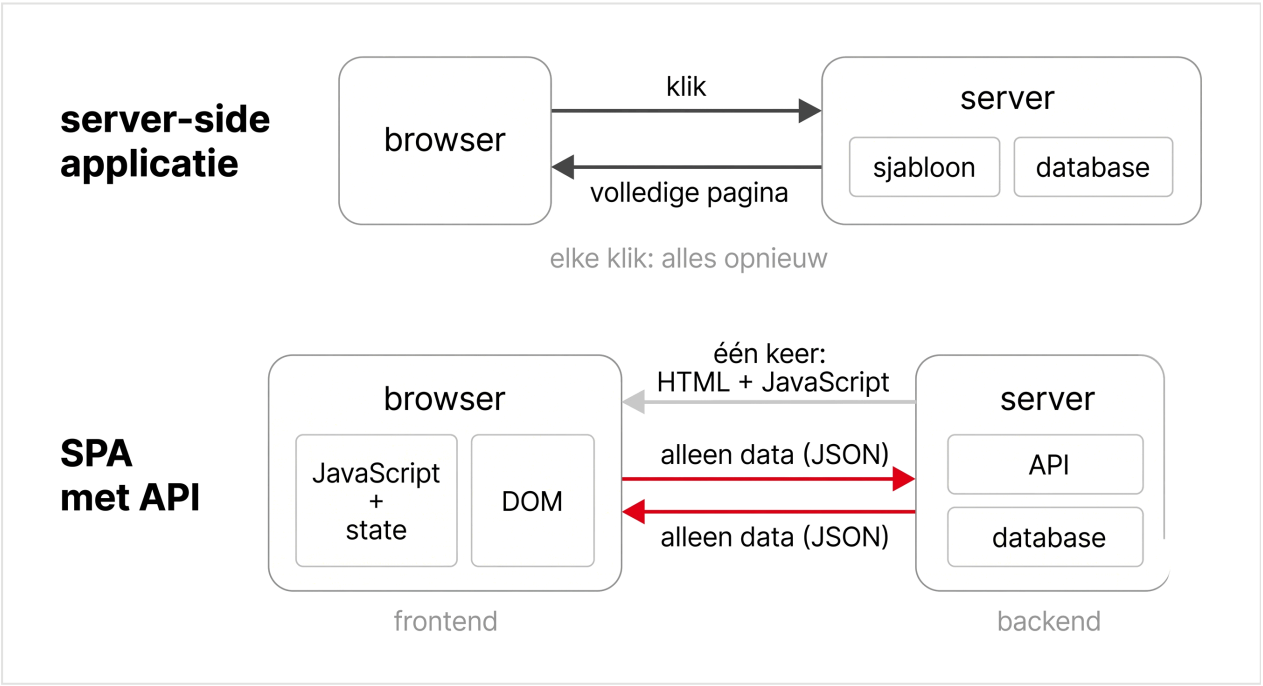
(SPA). De server levert één keer één HTML-pagina met een lading JavaScript, en vanaf dan neemt dat JavaScript het over. Navigeren, formulieren verwerken, gegevens tonen: het gebeurt allemaal in de browser. Met de server wordt alleen nog *data* uitgewisseld, als JSON, via een API.

Daarmee valt de applicatie in twee delen uiteen:

- de **frontend**: alles wat in de browser draait en wat de gebruiker ziet en aanraakt. In dit vak: Angular (hoofdstukken 2 tot 5)
- de **backend**: alles wat op de server draait. Gegevens bewaren, regels afdwingen, beslissen wie wat mag. In dit vak: Nest en MongoDB (hoofdstuk 6)

Ertussen ligt de **API**: de afspraak over welke gegevens de frontend kan vragen en sturen, en in welke vorm.

	SERVER-SIDE APPLICATIE	SPA MET API
Waar ontstaat de HTML?	op de server, per aanvraag	in de browser, door JavaScript
Wat gaat er over het netwerk?	volledige pagina's	alleen data (JSON)
Wat gebeurt er na een klik?	nieuwe pagina, alles herlaadt	een stuk van het scherm verandert
Waar leeft de "state", het geheugen van het scherm?	op de server (sessie)	in de browser
Wat doet de server?	alles	data bewaren en beslissen



Figuur 1.1 - Server-side applicatie: de server bouwt bij elke klik een volledige pagina. SPA met API: de browser krijgt één keer HTML en JavaScript en wisselt daarna alleen nog JSON uit.

Wat levert die splitsing op? Vier dingen, en ze verklaren waarom de industrie deze kant is opgegaan.

Een vlottere ervaring. Het scherm reageert meteen, want de code die reageert, zit al in de browser. Geen wit scherm tussen twee pagina's, geen formulier dat zijn inhoud verliest.

Schalen wordt goedkoper. Dit is het minst zichtbare maar belangrijkste voordeel. In een server-side applicatie bouwt de server voor elke gebruiker en bij elke klik een volledige pagina. Duizend gebruikers betekent duizend keer dat werk, op jouw server, op jouw kosten. In een SPA verhuist dat werk naar het toestel van de gebruiker: duizend gebruikers brengen duizend processoren mee. De frontend zelf bestaat na het bouwen uit gewone, statische bestanden. Die kan eender welke eenvoudige webserver uitdelen, of een CDN dat ze wereldwijd dicht bij de gebruiker zet, zonder dat er iets berekend wordt. De backend moet alleen nog kleine JSON-antwoorden leveren. En omdat de toestand van het scherm in de browser leeft, hoeft de server zich geen gebruikers te herinneren: je kan er bij drukte gewoon een tweede en een derde exemplaar naast zetten.

Eén backend, meerdere frontends. Dezelfde API bedient de webapplicatie, de mobiele app en het systeem van een partner. De regels staan op één plek.

Teams werken los van elkaar. Frontend en backend worden apart ontwikkeld, getest en online gezet. De API is het contract tussen beide. Zolang dat contract gerespecteerd wordt, kan elk team zijn eigen tempo volgen.

Nadelen? De eerste keer laden duurt langer, want de browser moet eerst de hele applicatie binnenhalen. Zoekmachines hebben het moeilijker met een pagina die leeg aankomt. Zonder JavaScript is er niets. Alles wat in de browser draait, is in handen van de gebruiker, dus de backend mag de frontend nooit vertrouwen (hoofdstuk 7). En vooral: de browser moet nu zelf bijhouden wat er op het scherm hoort te staan. Dat laatste is het probleem waar dit hele vak om draait. Maar eerst nog even uitzoomen.

Zes grote verschuivingen in dertig jaar

De splitsing in frontend en backend is geen plotse uitvinding. Het web is er in een zestal stappen naartoe gegroeid, en elke stap loste een probleem van de vorige op.

VANAF	VERSCHUIVING	WAT VERANDERT ER?	NAMEN
1991	Statische documenten	de server deelt bestanden uit; elke pagina is met de hand geschreven	HTML, later CSS (1996)
1995	Server-side applicaties	de server bouwt elke pagina op het moment van de aanvraag, uit een database	CGI, PHP, ASP, JSP; later Rails, Django, Express
2005	AJAX	een pagina haalt zelf gegevens op zonder te herladen	`XMLHttpRequest`, Gmail, Google Maps, jQuery
2010	De applicatie verhuist naar de browser	de SPA ontstaat; de server wordt een API; JavaScript draait ook buiten de browser	Backbone, AngularJS, Ember; Node.js
2013	Componenten en declaratieve UI	het scherm wordt een boom van herbruikbare componenten; je beschrijft *wat* er staat, niet *hoe* het er komt	React, Vue, Angular (2016); TypeScript
2020	De slinger zwaait terug, en wordt fijner	de server doet opnieuw mee voor snelheid en vindbaarheid; frameworks weten precies welk stukje scherm bij welk stukje data hoort	Next.js, Nuxt, Astro, HTMX; signals

Je kan deze tabel lezen als een slinger tussen server en client. Eerst deed de server alles, dan schoof het werk stap voor stap naar de browser, en de laatste jaren zoekt de sector het evenwicht. Dit vak staat met beide voeten in rij vier en vijf, gebruikt de reactiviteit van rij zes, en bouwt in hoofdstuk 6 de serverkant erbij. Sectie 1.5 zet de namen uit de laatste rij op hun plaats; sectie 1.7 vertelt hoe Angular zelf door deze tabel reisde.

Er is een zevende verschuiving bezig, en je zit er middenin: sinds 2022 schrijven AI-modellen een groeiend deel van alle code. Dat verandert niet wat een goede webapplicatie is. Het verandert wel wat jij moet kunnen (sectie 1.1).

Het synchronisatieprobleem

Terug naar de prijs van de SPA: de browser moet zelf bijhouden wat er op het scherm staat. Wat betekent dat concreet?

Elke applicatie heeft **state**: de gegevens zoals ze op dit moment zijn. De taken in een takenlijst. De ingelogde gebruiker. De inhoud van een winkelmandje. Welke filter er aanstaat. In je code is dat gewoon een variabele: een array, een object, een getal.

Het scherm toont die state. Maar het scherm ís die state niet. De DOM is een tweede, aparte voorstelling van dezelfde informatie, en de browser weet niet dat de twee iets met elkaar te maken hebben. Verander je de array, dan verandert er op het scherm niets. Iemand moet de DOM bijwerken. In vanilla JavaScript ben jij dat:

```
// app.js — vanilla: the data and the screen are two separate things
const taken = [];

function render() {
  const lijst = document.querySelector('#taken');
  lijst.innerHTML = '';
  for (const taak of taken) {
    const li = document.createElement('li');
    li.textContent = taak.titel;
    lijst.append(li);
  }
}

function voegToe(titel) {
  taken.push({ titel, gedaan: false });  // 1. change the data
  render();                              // 2. remember to update the screen!
}
```

Wat gebeurt hier? `render()` gooit de lijst op het scherm leeg en bouwt ze opnieuw op uit de array. `voegToe()` doet twee dingen na elkaar: de data aanpassen, en het scherm laten volgen. De techniek ken je uit Web Technology en Web Programming. Waar het om gaat, is stap twee.

Jij bent verantwoordelijk voor de afspraak dat het scherm de data volgt. Vergeet je één `render()`-aanroep - ergens diep in een event handler, drie weken later, in code van een collega - dan toont je applicatie oude informatie. Zonder foutmelding. Bij één lijstje hou je die discipline vol. Maar een echte applicatie heeft tientallen stukken state die op tientallen plaatsen zichtbaar zijn. De ingelogde gebruiker: naam in de header, rechten in het menu, knoppen die wel of niet mogen. Het winkelmandje: een badge met het aantal, het totaalbedrag, de knop om af te rekenen. Filters, zoektermen, laadstatussen. Elke wijziging moet op elke plaats aankomen, en het aantal combinaties explodeert.

Dit heet het **synchronisatieprobleem**: state en scherm met de hand gelijk houden. Het is dé bron van bugs in klassieke DOM-scripting, en het is exact het probleem dat elk modern framework oplost.

Onthoud dit, want het is de sleutel tot alles wat volgt: **een framework is in essentie een machine die het scherm automatisch synchroon houdt met je state.** De rest - componenten, routing, forms, tooling - zijn nuttige ondersteunende tools rond die kern.

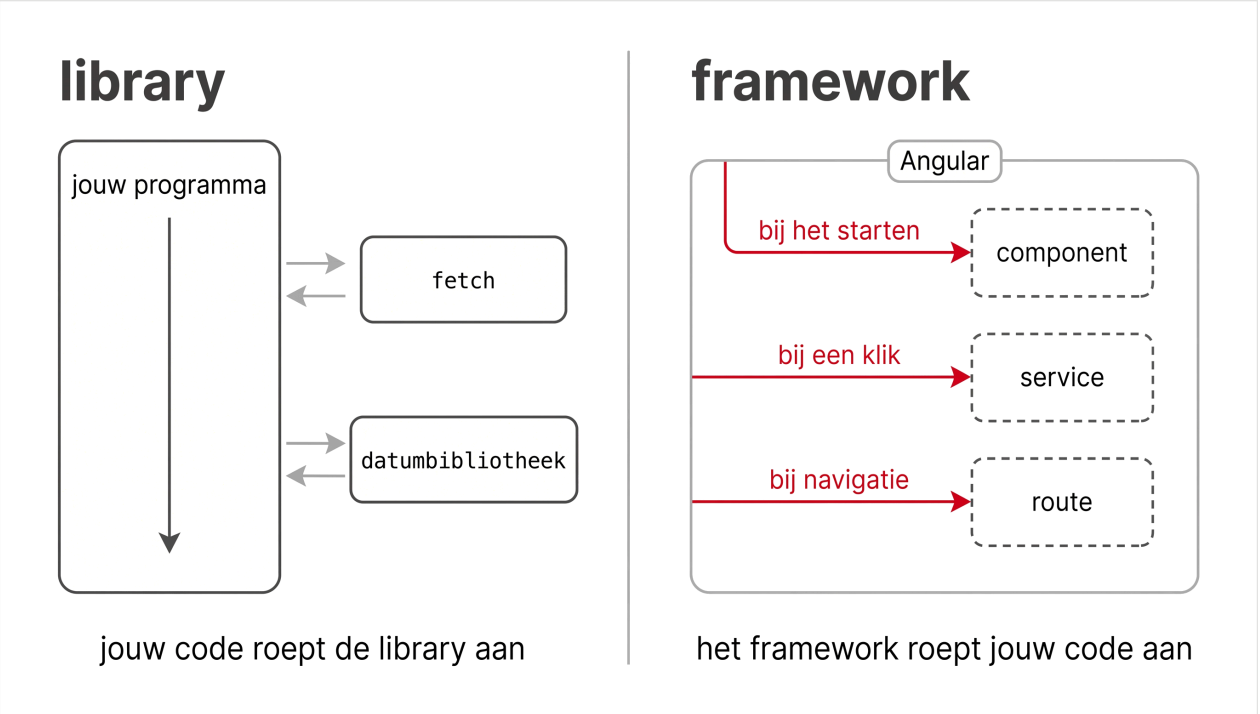
1.3 Wat een framework is (en wat niet)

Het woord "framework" wordt slordig gebruikt. We maken het precies met iets wat je kent.

Een **library** is code die jij aanroept. `fetch` roep je aan wanneer jij data wil; een datumbibliotheek roep je aan wanneer jij een datum wil formatteren. De controle ligt bij jou: jouw programma bepaalt de structuur, de library vult gaten.

Een **framework** draait die relatie om. Jij schrijft geen programma dat van boven naar beneden loopt en af en toe hulp inroept. Jij levert onderdelen aan volgens de spelregels van het framework - componenten, services, routes - en het *framework* beslist wanneer jouw code draait. Angular start de applicatie, bouwt jouw componenten op wanneer ze nodig zijn, roept jouw methodes aan wanneer de gebruiker klikt, en ruimt alles weer op. Dit principe heet **inversion of control** (letterlijk: omgekeerde controle). Het is even wennen. Je programma is geen verhaal meer met een begin en een einde, maar een verzameling antwoorden op vragen die het framework stelt. Jij beschrijft *wat* er moet bestaan; *wanneer* en *hoe vaak* is voor de machine.

	LIBRARY	FRAMEWORK
Wie roept wie aan?	jouw code roept de library aan	het framework roept jouw code aan
Wie bepaalt de structuur?	jij	het framework
Voorbeeld	`fetch`, een datumbibliotheek	Angular, Nest



Figuur 1.2 - Library: jouw programma roept `fetch` of een datumbibliotheek aan. Framework: Angular roept jouw component, service en route aan wanneer het dat nodig vindt.

Die verschuiving van *hoe* naar *wat* heeft een naam die je dit semester nog vaak hoort: **declaratief** programmeren, tegenover het **imperatieve** programmeren van je vanilla-code. Het verschil, met de takenlijst van daarnet:

```
<!-- Angular template (fragment): describe WHAT the list is -->
<ul>
  @for (taak of taken(); track taak.id) {
    <li>{{ taak.titel }}</li>
  }
</ul>
```

Dit is geen instructie ("maak een `li`, zet er tekst in, hang ze in de lijst") maar een *beschrijving*: de lijst toont de taken. Verandert de data, dan past het framework het scherm aan. Stap twee uit sectie 1.2 bestaat niet meer, en kan dus ook niet meer vergeten worden. De syntax (`@for`, `taken()`, `{{ }}`) leer je in hoofdstuk 2; nu telt het denkmodel: **de UI is een beschrijving, geen reeks handelingen.** Framework-ontwerpers zeggen het graag wiskundig: de UI is een functie van de state.

Wat krijg je er nog bij, behalve de automatische synchronisatie? Structuur. Een framework is *opinionated*: het heeft een mening over waar bestanden staan, hoe onderdelen heten, hoe data stroomt. Dat voelt als een beperking wanneer je alleen aan iets kleins werkt, en het is de reden waarom niemand een framework gebruikt voor een eenvoudige landingspagina. Maar zodra een applicatie groeit en er meerdere mensen aan werken - het normale geval in een bedrijf - worden die meningen veel waard. Elke Angular-applicatie ter wereld is op dezelfde manier georganiseerd, dus elke Angular-developer vindt zijn weg in elk Angular-project. Vergelijk dat met je vanilla-toepassingen van vorig jaar: hoe groter ze werden, hoe persoonlijker de structuur, en hoe moeilijker het werd om de code van iemand anders te lezen.

De prijs is een leercurve, en die van Angular is steil. Deze syllabus knipt ze in zeven stukken, maar verzwijgt ze niet: de komende weken leer je veel nieuwe begrippen tegelijk. Dat is **normaal**. De begrippen zijn bovendien niet Angular-specifiek. Componenten, declaratieve templates, reactiviteit, dependency injection, routing: elk modern framework is rond dezelfde concepten gebouwd. Wie Angular grondig begrijpt, leest en schrijft bvb. React binnen een week.

1.4 Geen magie: een reactief systeem in twintig regels

Voor we het landschap verkennen, ruimen we volgend misverstand op: het idee dat een framework iets *doet* wat jij niet zelf zou kunnen met JavaScript. De automatische synchronisatie van daarnet voelt als toveren: de data verandert en het scherm "weet" het. We bouwen die truc daarom meteen zelf, in vanilla JavaScript, met technieken die je uit Web Programming kent: closures en functies als waarden.

De vraag uit sectie 1.2 was: hoe kan het scherm weten dat de data veranderd is? Het antwoord: stop de data in een doosje waarvan we het lezen en het schrijven kunnen zien. Onthoud bij het lezen *wie* er las. Dan weet je bij het schrijven precies wie je moet verwittigen:

```
// signal.js — a complete reactive core, no framework involved
let actiefEffect = null;

export function signal(waarde) {
  const luisteraars = new Set();
  return {
    get() {
      if (actiefEffect) luisteraars.add(actiefEffect); // remember who reads
      return waarde;
    },
    set(nieuw) {
      waarde = nieuw;
      luisteraars.forEach(fn => fn()); // notify all readers
    },
  };
}
```

Dat is het doosje. Nu nog iemand die erin kijkt, in hetzelfde bestand:

```
// signal.js (continued)
export function effect(fn) {
  actiefEffect = fn;  // while fn runs, every get() sees who is asking
  fn();               // run once: this is where subscriptions happen
  actiefEffect = null;
}
```

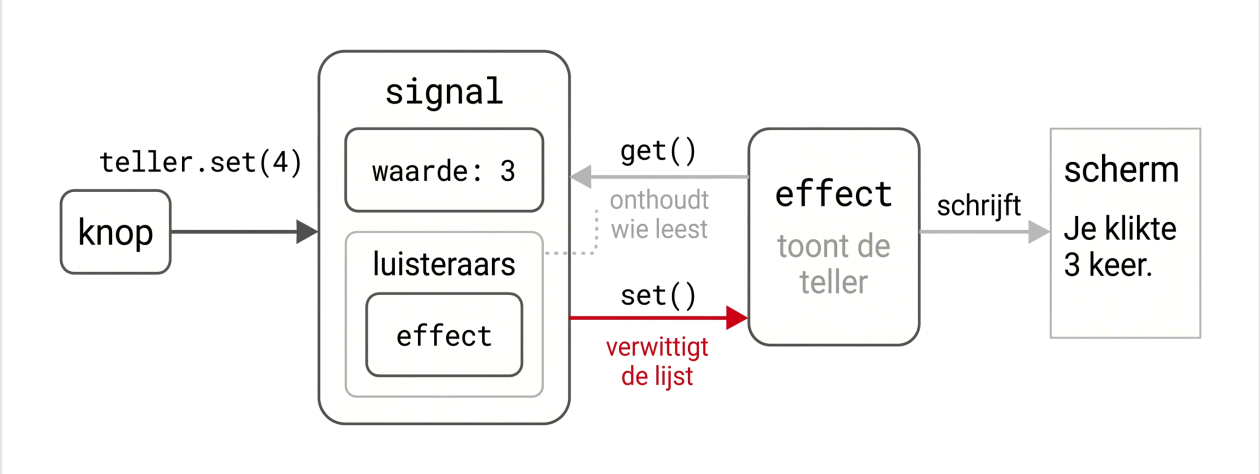
Twintig regels. Lees ze rustig, want hier gebeurt alles. Een `signal` is een closure rond een waarde. De `get` noteert wie er leest; de `set` roept iedereen op die lijst terug. Een `effect` is een functie die zich eerst aanmeldt (`actiefEffect`) en dan draait. Elke `get()` die ze onderweg doet, schrijft haar vanzelf in bij precies de signals waarvan ze afhangt. Gebruik het, en het probleem van sectie 1.2 is weg:

```
import { signal, effect } from './signal.js';

const teller = signal(0);

effect(() => {
  document.querySelector('#uit').textContent = `Je klikte ${teller.get()} keer.`;
});

document.querySelector('#knop').addEventListener('click', () => {
  teller.set(teller.get() + 1);   // change the data...
});                               // ...and that is ALL: the effect re-runs itself
```



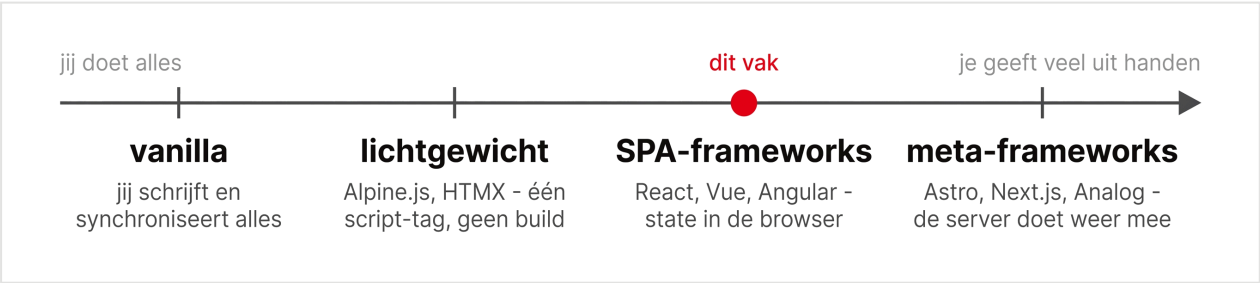
Figuur 1.3 - Een signal is een doosje met een waarde en een lijstje luisteraars. Het effect leest via `get()` en wordt onthouden; `set()` verwittigt de lijst, het effect draait opnieuw en schrijft naar het scherm.

Er is geen stap twee meer. De render-functie is een effect geworden dat zichzelf herhaalt wanneer zijn data verandert. Vergeten kan niet meer, want niemand moet er nog aan denken. In labo 1 bouw je dit zelf en herschrijf je er een applicatie mee. Doe dat aandachtig, want dit *is* het hart van modern Angular.

Wat het framework toevoegt aan deze twintig regels, is geen magie maar vakwerk: `computed` met caching, updates bundelen, effects opruimen wanneer een component verdwijnt, integratie met templates en compiler, en tien jaar aan afgehandelde randgevallen. Waardevol werk, maar werk van dezelfde soort als het jouwe. Denk hieraan wanneer Angular straks groot aanvoelt. Onderaan de stapel zit een doosje met een lijstje luisteraars, en dat doosje heb jij al eens gebouwd.

1.5 Het spectrum: van strooisel tot platform

"Framework of geen framework" is een valse tegenstelling. De echte vraag is: *hoeveel* van de verantwoordelijkheid geef je uit handen? Het antwoord is een spectrum waarop je elk gereedschap uit dit vakgebied kan plaatsen. Loop het één keer bewust af. Daarna herken je elke nieuwe naam als "ergens op deze as".



Figuur 1.4 - Het spectrum van vanilla over lichtgewicht frameworks en SPA-frameworks tot meta-frameworks. Dit vak staat bij de SPA-frameworks.

Helemaal links staat **vanilla**: jij schrijft alles en jij bent overal verantwoordelijk voor, ook voor de synchronisatie van sectie 1.2. Voor een site die vooral document is, met hier en daar wat interactie, is dat geen gebrek maar de juiste keuze.

Eén stap verder staan de **lichtgewicht frameworks**. Het mooiste voorbeeld is **Alpine.js**: één script-tag, geen build step, geen projectstructuur. Je zet declaratieve attributen op je bestaande HTML. Dit voorbeeld is volledig en werkt:

```
<script defer src="https://cdn.jsdelivr.net/npm/alpinejs@3/dist/cdn.min.js"></script>

<div x-data="{ taken: [], nieuw: '' }">
  <input x-model="nieuw" placeholder="Nieuwe taak">
  <button @click="taken.push(nieuw); nieuw = ''">Toevoegen</button>
  <ul>
    <template x-for="taak in taken">
      <li x-text="taak"></li>
    </template>
  </ul>
  <p x-show="taken.length === 0">Nog geen taken.</p>
</div>
```

State (`x-data`), two-way binding (`x-model`), event binding (`@click`), een lus (`x-for`), een conditie (`x-show`): dit zijn, tot in de naamgeving, de concepten die je in hoofdstuk 2 in Angular leert. Dat is geen toeval. Het zijn de gedeelde grondbegrippen van declaratieve UI, en Alpine bewijst dat ze niet aan grootte gebonden zijn. Waar Alpine bewust stopt: geen componenten met contracten, geen types, geen routing, geen tooling, geen structuur voor tien schermen en drie developers. Het is gemaakt om een pagina die van de server komt *strooisels* van interactie te geven, en daarin is het uitstekend.

In dezelfde gewichtsklasse, maar met een heel andere filosofie, staat **HTMX**. Alles wat we dit semester doen, draait rond state in de browser: de SPA-gedachte. HTMX draait de klok bewust terug. De *server* blijft de baas over de HTML. Attributen als `hx-get` en `hx-target` laten een element bij interactie een *fragment HTML* ophalen en op zijn plaats zetten. Geen JSON, geen state in de browser, geen JavaScript van jouw hand:

```
<!-- fragment: needs a server that returns HTML snippets -->
<button hx-get="/taken/lijst" hx-target="#lijst">Ververs</button>
<div id="lijst"><!-- server-rendered rows land here --></div>
```

Deze *hypermedia*-beweging is een ernstige en soms terechte kritiek op de dominantie van de SPA: voor sites die in wezen documenten zijn met interactieve stukken, is een complete applicatie in de browser overkill. Je komt de aanpak tegen onder de naam **AHA-stack**: Astro, HTMX, Alpine. Waarom bouwt dit vak er dan niet mee? Om twee redenen. Principieel: HTMX lost een ander probleem op. Zodra een applicatie veel state in de browser heeft - formulieren met live validatie, filters, een winkelmandje, alles waar dit vak naartoe werkt - moet die state *ergens* beheerd worden, en "op de server" betekent dan een rondreis per toetsaanslag. Praktisch: HTMX heeft een server

nodig die fragmenten maakt, en die bouw je pas in hoofdstuk 6. Onthoud vooral de denkles: **niet elke interactieve site hoort een SPA te zijn.** Wie dat weet, maakt straks betere keuzes dan wie maar één hamer heeft.

Rechts van het midden staan de **SPA-frameworks**: React, Vue, Angular. Daarover meer in sectie 1.6. Helemaal rechts staan de **meta-frameworks**, die bovenop een SPA-framework de serverkant terugbrengen: **Astro** (content eerst, met "eilanden" van interactiviteit), Next.js voor React, Nuxt voor Vue, Analog voor Angular. Zij maken pagina's op de server voor snelheid en vindbaarheid en laten de browser bijspringen waar het moet. Dat is de laatste rij uit de tabel van sectie 1.2: het document en de applicatie groeien weer naar elkaar toe. Voor de applicaties van dit vak - dashboards, beheertools, apps achter een login - is klassieke client-side rendering het juiste antwoord, en daar houden we het bij.

De vraag "welk punt op het spectrum?" is daarmee een echte engineeringbeslissing, met echte criteria. Hoeveel state heeft dit product in de browser? Hoeveel schermen? Hoe lang moet het meegaan? Hoeveel mensen werken eraan? Een portfoliosite in Angular bouwen is met een vrachtwagen naar de bakker rijden. Een bankapplicatie op losse sprinkles bouwen is de vrachtlading op een bakfiets stapelen. Na dit semester kan je die afweging maken én benoemen. Dat is kennis die in het post-AI-tijdperk uiterst belangrijk is geworden.

1.6 De grote drie

Op het applicatie-uiteinde van het spectrum domineren drie spelers al bijna tien jaar. Je moet ze alle drie kunnen situeren, al was het maar omdat je in sollicitatiegesprekken en op de werkvloer over "framework-keuze" zal horen discussiëren alsof het een geloofskwestie is.

React (Meta, 2013) noemt zichzelf een library, geen framework. Het regelt de UI-laag en laat routing, forms en tooling over aan een ecosysteem van losse pakketten. Dat geeft veel vrijheid en veel keuzestress; twee React-projecten kunnen er totaal verschillend uitzien. React is op dit moment met afstand het populairst.

Vue (2014) is het progressieve midden. Je kan het als lichte library in een bestaande pagina gebruiken én als volwaardig framework. Het is geliefd om zijn lage instapdrempel.

Angular (Google, 2016) is het volledige platform: framework, router, forms, HTTP-client, testtooling en CLI uit één hand, in TypeScript, met duidelijke conventies. Waar React kiest voor vrijheid, kiest Angular voor samenhang. Dat maakt het het huisframework van organisaties waar applicaties groot worden en lang meegaan: banken, overheden, industrie. Ook en juist in België.

Belangrijker dan de verschillen is wat je ziet als je uitzoomt: de drie zijn naar elkaar toe gegroeid. Alle drie werken ze met componenten. Alle drie beschrijven ze de UI declaratief. En alle drie zijn ze, elk in hun eigen tempo, uitgekomen bij fijnmazige reactiviteit als kern: Vue met refs, Solid en Angular met signals, React via zijn compiler. Jij weet sinds sectie 1.4 wat daaronder zit: het doosje met het lijstje luisteraars. Wat je dit semester leert, is dus niet strikt Angular-specifiek maar de gedeelde grammatica van moderne frontend.

Dat is ook het antwoord op de vraag "waarom leren we niet gewoon React, dat vraagt de markt toch meer?". Een opleiding traint geen syntax maar modellen. Angular is door zijn structuur en zijn volledigheid het best leerbare en toetsbare voertuig voor die modellen. De vaardigheid "een nieuw framework oppikken" krijg je er daarna bij, en die heb je in je loopbaan vaker nodig dan eender welke framework-kennis.

Eén randnoot. Het server-first-idee van sectie 1.5 bestaat ook bínnen Angular, als ingebouwde server-side rendering. Je krijgt de vraag letterlijk bij het aanmaken van je eerste project. Je antwoordt nee, om de redenen die je nu kan uitleggen.

1.7 Angular vandaag - en waarom dat woord "vandaag" ertoe doet

Voor de meeste vakken is de geschiedenis van hun gereedschap een voetnoot. Voor dit vak is ze praktisch nodig. In de bronnen die jij zal raadplegen, zijn er drie Angulars in omloop, en twee ervan moet je herkennen om ze te vermijden.

Het begon in 2010 met **AngularJS**, een van de eerste grote frameworks (rij vier in de tabel van sectie 1.2). Vernieuwend voor zijn tijd, maar architecturaal een doodlopende straat. Google herschreef het van nul en bracht dat in 2016 uit als **Angular 2**: een compleet ander framework dat alleen de naam deelde. Sindsdien verschijnt er elk half jaar een major versie, voorspelbaar en met een migratiepad. De versie in deze syllabus is **Angular 22**, van juni 2026.

Binnen dat "tweede Angular" begon vanaf versie 16 (2023) een grondige vernieuwing die de community de **Angular Renaissance** noemt. In drie jaar verdween bijna alle beruchte boilerplate. NgModules, een verplichte administratieve laag rond elke component, werden overbodig. Templates kregen een leesbare ingebouwde control flow. En de grootste verandering: **signals** werden het reactiviteitsmodel, de machine waarmee Angular precies weet welk stukje scherm moet meebewegen met welk stukje state. Daarna werd dat nieuwe model de standaard. Sinds versie 21 draaien nieuwe projecten *zoneless*: zonder de oude detectiemachine die bij elke gebeurtenis de hele applicatie controleerde. Sinds versie 22 staat change detection standaard op de zuinige stand. Wat die termen precies betekenen, leer je in hoofdstuk 2. Onthoud nu dit: **het Angular van deze syllabus is het Angular van na de Renaissance**, slanker en eenvoudiger dan wat de studenten voor jou moesten leren.

Waarom doet dit ertoe? Omdat het internet het nog niet weet. Tutorials, antwoorden op Stack Overflow, YouTube-cursussen en de trainingsdata van AI-modellen dateren grotendeels van voor of midden in die overgang. Jouw project bevat geen `app.module.ts`, terwijl elke oudere tutorial erover begint. Een model stelt je `*ngIf` voor waar jouw Angular `@if` schrijft.

Vuistregel: jouw referentie is angular.dev - de officiële documentatie, die wél mee is - en deze syllabus. Al de rest lees je met argwaan en met een blik op de datum.

1.8 TypeScript: types als contract

Angular-code schrijf je niet in JavaScript maar in **TypeScript**. Dat is geen detail van de tooling maar een pijler van het hele systeem. De taal zelf is herhaling: je werkte er in Web Programming al mee. Elke geldige regel JavaScript is geldige TypeScript, plus **types**. Dit vak voegt geen nieuwe syntax toe maar een scherper denkmodel. In een framework-codebase houden types alles bijeen, en je moet precies weten wat ze wél garanderen en wat niet.

Begin bij het soort fout dat types bestrijden. JavaScript vraagt zich pas op het allerlaatste moment af of een operatie kan. Je kan maandenlang een functie hebben die `gebruiker.name` leest terwijl de data `gebruiker.naam` bevat. Je ontdekt het pas wanneer een gebruiker van je toepassing `undefined` op het scherm ziet. In TypeScript beschrijf je de vorm van je data, en de compiler controleert *voor de code ooit draait* of elke regel die vorm respecteert:

```
interface Gebruiker {
  id: number;
  naam: string;
  email: string;
}

const gebruiker: Gebruiker = { id: 1, naam: 'Barbara', email: 'barbara@ap.be' };
console.log(gebruiker.name);
// error TS2339: Property 'name' does not exist on type 'Gebruiker'.
```

Een `interface` is een **contract**: een afspraak over hoe data eruitziet, die de compiler afdwingt bij iedereen die de data aanraakt. Dat verschuift een hele klasse fouten van runtime (bij de gebruiker, 's avonds, onvindbaar) naar compile time (in je editor, meteen, met een rode kringel onder de fout). En het voedt de tooling die je vanaf deze week dagelijks gebruikt: de autocomplete, de foutmeldingen en de refactoring in VS Code zijn toepassingen van het typesysteem.

Drie eigenschappen van dat systeem moet je nu al kennen, omdat ze de rest van de syllabus kleuren.

Ten eerste: **TypeScript is lui op de goede manier.** Je moet niet overal types schrijven; de compiler leidt ze af waar hij kan (*type inference*). `const prijs = 9.99` heeft type `number`, zonder dat jij iets deed. Je schrijft types expliciet waar ze een contract vormen (functiesignaturen, datamodellen, publieke API's) en laat inference het werk doen in de rest.

Ten tweede: **TypeScript kijkt naar vorm, niet naar naam.** Een object moet niet verklaren dat het een `Gebruiker` is. Als het de juiste velden met de juiste types heeft, *is* het er één (*structural typing*). Dat maakt het typesysteem soepel op de plaats waar webapplicaties het nodig hebben: JSON die van een API binnenkomt, heeft nooit een naamkaartje, maar wel een vorm.

Ten derde, en dit is de belangrijkste beperking: **types bestaan alleen tijdens het compileren.** Browsers spreken geen TypeScript. Je code wordt voor uitvoering vertaald naar gewoon JavaScript. Dat heet *transpiling*, en de Angular CLI doet het onzichtbaar voor jou, met *source maps* zodat je bij het debuggen je eigen TypeScript ziet. Bij die vertaling verdwijnen alle types. Een `interface`

beschermt je dus tegen je eigen typfouten, maar niet tegen een API die andere data terugstuurt dan beloofd. Het contract wordt gecontroleerd bij het schrijven, niet bij het uitvoeren. Onthoud dat verschil; het verklaart in hoofdstuk 6 waarom de backend voor validatie ander gereedschap nodig heeft dan een interface.

Meer TypeScript dan dit heb je als startkapitaal niet nodig. De rest van de taal - generics zoals `Product[]`, union types zoals `'asc' | 'desc'`, optionele velden met `?` - halen we op wanneer de code erom vraagt, telkens met een korte duiding.

1.9 Het Node-ecosysteem: de werkplaats

In Web Programming leerde je Node.js kennen als runtime om JavaScript op een server te draaien. In dit vak krijgt Node er een tweede rol bij: **de werkplaats van de frontend-developer.** De applicatie die je bouwt, draait straks in de browser van de gebruiker. Maar het gereedschap waarmee je haar bouwt, test en bundelt, draait op Node, op jouw laptop. De Angular CLI is een Node-programma. De development server is een Node-programma. De TypeScript-compiler is een Node-programma.

Vandaar een harde vereiste voor dit vak: Angular 22 weigert oudere Node-versies. Je hebt minstens Node 22.22 nodig, of 24.15 en hoger. **Vuistregel:** installeer de huidige LTS-versie (Node 24) en je zit goed.

Met Node komt **npm**, en met npm het bestand dat je kent uit je Express-projecten van vorig jaar: `package.json`. Nieuw is het niet. Wat in dit vak telt, is dat je dit paspoort van eender welk JavaScript-project vlot kan *lezen*. Drie onderdelen zijn belangrijk:

```
{
  "name": "eerste-app",
  "scripts": {
    "start": "ng serve",
    "build": "ng build"
  },
  "dependencies": { "@angular/core": "^22.1.0" },
  "devDependencies": { "@angular/cli": "^22.1.8" }
}
```

De `dependencies` zijn de pakketten die je applicatie nodig heeft om te *draaien*. De `devDependencies` zijn alleen nodig om te *bouwen*; de gebruiker van je app heeft niets aan de CLI. De `scripts` zijn afspraken over commando's: `npm start` werkt in elk JavaScript-project ter wereld, wat er ook achter zit. En het dakje in `^22.1.0` is **semantic versioning**: "versie 22.1 of hoger, maar binnen 22". Major versies (22 naar 23) mogen breken, minors en patches niet. Angular hanteert dezelfde afspraak voor zijn halfjaarlijkse releases.

Wat er *niet* in het paspoort staat maar wel in je projectmap verschijnt: `node_modules`, de map waarin npm de dependencies (en hún dependencies, en zo verder) installeert. Tienduizenden bestanden, honderden megabytes. Twee regels volstaan om er nooit last van te hebben.

Eén: `node_modules` is wegwerpmateriaal. De map staat in `.gitignore`, gaat nooit mee in een zip of een repository, en wordt op elke nieuwe machine opnieuw opgebouwd met één commando: `npm install`. Dat commando leest `package.json` en zet exact klaar wat er moet staan, met `package-lock.json` als gedetailleerde pakbon.

Twee: zie je een project zónder `node_modules` - een gedownload voorbeeld, een gekloonde repo, de basisapps van dit vak - dan is `npm install` altijd je eerste commando.

1.10 Je gereedschap: de Angular CLI en je eerste applicatie

Een Angular-project maak je niet met de hand. Je laat het genereren, starten, uitbreiden en bouwen door één programma: de **Angular CLI** (command-line interface), in de terminal het commando `ng`. Wie vlot met de CLI werkt, werkt vlot met Angular. Labo 2 draait er volledig rond.

De werkplaats inrichten

Controleer eerst wat er staat:

```
node -v    # v22.22 or higher; v24 (LTS) is the safe choice
npm -v     # comes with Node
```

Te oud of afwezig? Installeer de LTS-versie via nodejs.org. Installeer daarna de Angular CLI, eenmalig en globaal (`-g`), zodat het commando `ng` overal beschikbaar is:

```
npm install -g @angular/cli
ng version
```

Je editor blijft VS Code, met één verplichte uitbreiding: de extensie **Angular Language Service**. Zij brengt het typesysteem van sectie 1.8 naar je templates: autocomplete, foutmeldingen en documentatie, ook in HTML-bestanden. Werken zonder is werken met een blinddoek.

`ng new`: een project genereren

Ga in de terminal naar de map waarin je je projecten bewaart en genereer een applicatie:

```
ng new eerste-app
```

De CLI stelt een paar vragen. Kies **CSS** als stylesheet-formaat. Antwoord **nee** op server-side rendering (sectie 1.5 en 1.6 legden uit waarom). Vraagt de CLI of ze configuratie voor AI-tools moet aanmaken, kies dan **none**. Daarna zie je een lijst `CREATE`-regels voorbijkomen - elk bestand dat de CLI voor jou aanmaakt - en loopt er een `npm install` van een minuut. Je weet ondertussen wat daar gebeurt.

`ng serve`: de werklus

Start de development server:

```
cd eerste-app
ng serve
```

Open `http://localhost:4200`. Daar staat ze: jouw eerste Angular-applicatie. Een lokale server zoals Live Server vorig jaar, maar één die je code onderweg compileert. Laat hem draaien, open `src/app/app.html`, verander wat tekst en bewaar. De browser ververst zichzelf. Dat is de werklus voor de rest van het semester: bewerk, bewaar, kijk. Stoppen doe je met Ctrl-C in de terminal.

Tip: `ng serve --open` opent de browser voor jou. `ng serve --port 4300` kiest een andere poort.

Rondleiding: wat de CLI voor je maakte

`ng new` maakt een twintigtal bestanden. Dat lijkt veel, maar ze vallen in vier groepen, en met één groep heb je dagelijks te maken.

Het project als geheel (in de hoofdmap):

BESTAND OF MAP	ROL
`package.json`, `package-lock.json`	het paspoort en de pakbon (sectie 1.9)
`node_modules/`	de geïnstalleerde pakketten; wegwerpmateriaal
`angular.json`	configuratie van de CLI zelf: hoe bouwen, hoe serveren, waar staat wat
`tsconfig.json`	instellingen van de TypeScript-compiler, streng afgesteld; zo houden we het
`.gitignore`	wat niet in je repository komt; `node_modules` en `dist` staan er al in

De ingang van de applicatie (in `src/`):

BESTAND	ROL
`src/index.html`	de ene pagina van je Single-Page Application
`src/main.ts`	het startpunt: hier geeft jouw code de controle aan Angular
`src/styles.css`	globale stijlen, voor de hele applicatie

Jouw domein (in `src/app/`): hier leeft alles wat je de komende weken schrijft.

BESTAND	ROL
`app.ts`	de rootcomponent: de klasse
`app.html`	de rootcomponent: de template
`app.css`	de rootcomponent: de stijlen, alleen voor deze component
`app.spec.ts`	een testbestand; laat het staan
`app.config.ts`	applicatiebrede instellingen; je hebt het nodig vanaf hoofdstuk 3
`app.routes.ts`	de routetabel, nu nog leeg; hoofdstuk 3

Wat je mag negeren: `public/` (statische bestanden zoals het favicon), `.vscode/`, `.editorconfig`, `.prettierrc`, `README.md`, en de twee varianten `tsconfig.app.json` en `tsconfig.spec.json`. Ze zijn nuttig, maar je moet er dit semester niets in veranderen.

Twee bestanden verdienen een tweede blik. `index.html` is bijna leeg. In de body staat één tag die in HTML niet bestaat:

```
<!-- src/index.html (fragment) -->
<body>
  <app-root></app-root>
</body>
```

En toch zie je in de browser een volledige pagina. Dat is het SPA-model uit sectie 1.2, live: alles wat je ziet, is door JavaScript opgebouwd. Vergelijk in de browser *View Page Source* (wat de server stuurde) met de *Elements*-tab in DevTools (wat er nu staat).

`main.ts` bevat één betekenisvolle aanroep:

```
// src/main.ts
bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

Dat is inversion of control (sectie 1.3) in één regel. Jouw programma "start" niet. Het overhandigt zijn rootcomponent en zijn configuratie aan het framework, en Angular neemt het over.

Een eerste blik op een component

Die rootcomponent is het eerste stuk Angular dat je leest. Open `src/app/app.ts`:

```ts
// src/app/app.ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  imports: [RouterOutlet],
  selector: 'app-root',
  styleUrl: './app.css',
  templateUrl: './app.html',
})
export class App {
  protected readonly title = signal('eerste-app');
}
```

Wat staat hier? Een gewone TypeScript-klasse, `App`, met één veld. Dat veld is een `signal`: het doosje uit sectie 1.4, nu dat van Angular. Boven de klasse staat `@Component({...})`, een etiket dat Angular vertelt dat dit een **component** is en wat erbij hoort. De `selector` is de tagnaam, en die herken je: `<app-root>` uit `index.html`. `templateUrl` en `styleUrl` wijzen naar de HTML en de CSS van deze component. In `imports` staat wat de template van deze component zelf gebruikt.

Een component is dus een stuk scherm met drie onderdelen: een klasse (wat het weet en kan), een template (wat er staat) en stijlen (hoe het eruitziet). Een Angular-applicatie is een boom van zulke componenten, met `App` als root. Meer moet je nu niet weten. Hoofdstuk 2 gaat in de diepte.

`app.html` bevat bij een vers project honderden regels welkomstpagina. Gooi alles weg behalve de laatste regel, `<router-outlet />`, en zet er je eigen HTML boven. Zoek voor je dat doet eens naar `{{ title() }}`: daar leest de template het signal uit de klasse.

`ng generate`: nieuwe onderdelen maken

Nieuwe componenten typ je niet zelf bij elkaar. Je laat ze genereren:

```sh
ng generate component begroeting
# short form:
ng g c begroeting
```

```
CREATE src/app/begroeting/begroeting.css (0 bytes)
CREATE src/app/begroeting/begroeting.spec.ts (560 bytes)
CREATE src/app/begroeting/begroeting.ts (201 bytes)
CREATE src/app/begroeting/begroeting.html (25 bytes)
```

Een eigen map met dezelfde vier bestanden als de rootcomponent, en de selector `app-begroeting`. De component bestaat nu, maar staat nog nergens op het scherm. Daarvoor zijn twee stappen nodig, en je zal ze dit semester honderden keren zetten. Eén: zet de tag in de template van de ouder. Twee: importeer de klasse bij die ouder.

```
<!-- src/app/app.html -->
<app-begroeting />
<router-outlet />
```

```
// src/app/app.ts (fragment)
import { Begroeting } from './begroeting/begroeting';

@Component({
  imports: [RouterOutlet, Begroeting],
  // ...selector, styleUrl, templateUrl...
})
```

Vergeet je stap twee, dan weigert de compiler met NG8001 (F2.3 in hoofdstuk 2). Labo 2 laat je die fout één keer bewust maken.

De CLI kan meer genereren dan componenten: services, pipes, guards, interfaces. Ze komen aan bod wanneer de syllabus ze nodig heeft. De commando's en opties die je vanaf nu moet kennen:

COMMANDO	WAT HET DOET
`ng new <naam>`	een nieuw project genereren
`ng serve`	de development server starten (`--open`, `--port 4300`)
`ng generate component <naam>`	een component genereren; kort: `ng g c <naam>`
`ng g c <naam> --dry-run`	tonen wat er zou gebeuren, zonder iets te schrijven
`ng build`	de applicatie bouwen voor productie, in `dist/`
`ng test`	de tests uitvoeren
`ng version`	versies van CLI, Node en Angular tonen
`ng help`, `ng <commando> --help`	de ingebouwde handleiding

Vuistregel: twijfel je wat een commando gaat doen, zet er `--dry-run` achter. Weet je een optie niet meer, vraag het met `--help`. Niemand kent de CLI uit het hoofd; wel weet iedereen dat ze zichzelf documenteert.

`ng build`: klaar voor een echte server

Eén commando nog. `ng build` maakt in `dist/` de geoptimaliseerde, gebundelde versie die je op een echte server zet:

```
Initial chunk files | Names         |  Raw size | Estimated transfer size
main-S57UDMEY.js    | main          | 193.06 kB |                52.80 kB
styles-5INURTSO.css | styles        |   0 bytes |                 0 bytes
```

Kijk naar de getallen. Je `node_modules` weegt bijna 300 MB; wat de gebruiker downloadt, is een goede 50 kB. De inhoud van `dist/eerste-app/browser/` is statisch: HTML, JavaScript en CSS. Dat zijn de bestanden uit sectie 1.2 die eender welke eenvoudige webserver of een CDN kan uitdelen. Hoe je een Angular-app online zet, komt aan bod zodra er iets te tonen valt.

1.11 De volledige stack: waar dit vak naartoe werkt

Nu de omgeving draait, hoort er een routekaart bij, want dit vak stopt niet bij de browser.

Een SPA zonder backend is een demo. Echte applicaties bewaren hun data centraal, achter een API: de frontend vraagt en toont, de backend beslist en bewaart (sectie 1.2). Dat is voor jou geen onbekend terrein, want in Web Programming bouwde je zelf een Express-server met MySQL erachter. In de tweede helft van dit vak zet je die kennis om naar het moderne equivalent.

Nest.js is een server-framework op dezelfde Node-fundamenten, maar bewust naar Angular gemodelleerd: dezelfde manier van opdelen, dezelfde services, dezelfde dependency injection. Je zal het gevoel hebben dat je het al half kent. **MongoDB** is een database die JSON-achtige documenten bewaart, zodat data van database tot scherm één vorm houdt. In hoofdstuk 6 leer je ze naast het MySQL van vorig jaar plaatsen. Alles is TypeScript, van het eerste formulierveld tot de databasequery.

Wie oudere bronnen leest, komt voor die combinatie de term **MEAN-stack** tegen: MongoDB, Express, Angular, Node. Het idee erachter - één taal, JSON van kop tot staart - leeft voort. De E van Express is in moderne stacks vervangen door Nest, dat er bovenop bouwt.

De route ligt nu open. Eerst word je goed in de frontend (hoofdstukken 2 tot 5), dan bouw je er een backend achter (hoofdstuk 6), en tot slot werk je het geheel professioneel af (hoofdstuk 7).

Onder de motorkap: wat `ng serve` echt doet

Je typt `ng serve`, een seconde later is `localhost:4200` een werkende applicatie. In die seconde gebeurt meer dan in de rest van dit hoofdstuk samen. Wie het wil weten, leest hier mee.

Eerst leest de CLI `angular.json` en `tsconfig.json`: wat moet er gebouwd worden en volgens welke regels. Dan begint het eigenlijke werk. Elk `.ts`-bestand wordt door de TypeScript-compiler gecontroleerd (hier vallen je contractbreuken uit sectie 1.8 door de mand) en vertaald naar JavaScript. Je templates ondergaan hetzelfde lot, en dat verrast de meeste studenten. Angular-templates zijn geen HTML die in de browser "geïnterpreteerd" wordt. De **Angular-compiler** vertaalt ze naar JavaScript-instructies die de DOM rechtstreeks opbouwen. Wat de browser ontvangt, bevat jouw `@for` dus niet meer; die bestaat alleen voor jou en de compiler. Templates zijn daardoor snel én controleerbaar. De Language Service die je typfout in een template aanwijst, is dezelfde compiler in een ander jasje.

Daarna bundelt **esbuild**, een zeer snelle bundler, de duizenden losse modules - jouw code plus alles uit `node_modules` wat je echt gebruikt - tot een handvol JavaScript-bestanden. De dev server zet verwijzingen daarnaar in je lege `index.html`. Hij houdt via een WebSocket een lijntje open met je browser: bewaar je een bestand, dan hercompileert hij alleen wat veranderde en krijgt de browser het sein om te verversen. Daarom duurt de eerste start een seconde en is elke wijziging daarna bijna onmiddellijk zichtbaar.

`ng build` doorloopt dezelfde keten en voegt optimalisaties toe die voor een dev server te traag zijn. *Minification*: alle namen kort, alle witruimte weg. *Tree-shaking*: ongebruikte code wordt weggelaten. Dat is de reden dat een Angular-bundel klein blijft terwijl `node_modules` gigantisch is. Een SPA heeft geen bijzondere server nodig; alle intelligentie zit in de bestanden zelf.

Veelgemaakte fouten

F1.1 - `ng: command not found` na installatie. *Symptoom*: npm installeerde de CLI zonder klagen, maar `ng` bestaat niet. *Oorzaak*: de map waarin npm globale commando's plaatst, staat niet in het PATH van je terminal, of je terminal stond nog open van voor de installatie. *Remedie*: sluit de terminal volledig en open een nieuwe; controleer met `ng version`. Blijft het probleem, installeer Node opnieuw via de officiële installer van nodejs.org (die zet het PATH goed) en daarna de CLI.

F1.2 - PowerShell weigert `ng`. *Symptoom*: op Windows antwoordt PowerShell met `ng.ps1 cannot be loaded because running scripts is disabled on this system`. *Oorzaak*: het standaardbeleid van PowerShell laat geen scripts toe, en `ng` is er één. *Remedie*: voer één keer `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` uit, zoals angular.dev het voorschrijft, en open een nieuwe terminal.

F1.3 - De CLI weigert je Node-versie. *Symptoom*: `The Angular CLI requires a minimum Node.js version of v22.22.3 or v24.15.0`. *Oorzaak*: je Node dateert van vorig jaar (sectie 1.9). *Remedie*: installeer de huidige LTS via nodejs.org, open een nieuwe terminal, controleer met `node -v`.

F1.4 - `ng serve` buiten een project. *Symptoom*: `This command is not available when running the Angular CLI outside a workspace`. Het commando dat gisteren werkte, weigert vandaag. *Oorzaak*: je staat in de verkeerde map. `ng serve` werkt alleen in een map met een `angular.json`, of eronder. *Remedie*: `cd` naar je projectmap en controleer of je `angular.json` ziet. Klassieker: je staat één niveau te hoog, in de map die je projecten bevat.

F1.5 - Een gedownload project start niet. *Symptoom*: je opent een basisapp of het project van een medestudent en alles is rood: `Cannot find module '@angular/core'`. *Oorzaak*: `node_modules` ontbreekt. Terecht, want die map hoort niet meegestuurd te worden (sectie 1.9). *Remedie*: `npm install` in de projectmap, dan pas `ng serve`. Omgekeerd: stuur nooit `node_modules` mee. Je zip wordt er duizend keer groter van en niemand heeft er iets aan.

F1.6 - "Waar is app.module.ts?" *Symptoom*: de tutorial, de slides van vorig jaar of het AI-antwoord verwijst naar bestanden en begrippen (`app.module.ts`, `NgModule`, `declarations`) die in jouw project niet bestaan. *Oorzaak*: de bron beschrijft Angular van voor de Renaissance (sectie 1.7). Jouw project is correct; de bron is oud. *Remedie*: leg de bron weg en zoek het onderwerp op angular.dev, of zoek in appendix B het moderne equivalent. Probeer nooit je project "compatibel te maken" met een oude tutorial.

F1.7 - Poort 4200 is al bezet. *Symptoom*: `Port 4200 is already in use.` *Oorzaak*: er draait nog een `ng serve`, in een vergeten terminalvenster of na een crash. *Remedie*: stop de oude server (Ctrl-C in dat venster) of start op een andere poort: `ng serve --port 4300`. Werk nooit met twee servers op dezelfde app zonder het te weten; je kijkt dan ooit naar de verkeerde.

F1.8 - Wijzigingen verschijnen niet in de browser. *Symptoom*: je bewerkt en bewaart, maar het scherm blijft oud. *Oorzaak*: meestal banaal. Het bestand is niet bewaard, de compiler vond een fout, of je kijkt naar een ander tabblad of een andere poort. *Remedie*: de terminal is je eerste blik, niet de browser. Een rode compilatiefout in de terminal betekent: je krijgt je wijziging pas te zien als de fout weg is.

Met AI aan je zij: gedateerd Angular herkennen

Vraag een model om "een simpele Angular-app" en de kans is reëel dat je code krijgt die drie jaar oud is. Niet omdat het model dom is, maar omdat zijn trainingsdata de geschiedenis van sectie 1.7 weerspiegelt: er is gewoon veel meer oud Angular geschreven dan nieuw. Na dit hoofdstuk kan je de snelste controles al zelf doen. Zie je één van deze patronen, dan lees je gedateerde code:

- een bestand `app.module.ts` of de decorator `@NgModule({...})`
- `*ngIf` of `*ngFor` in templates (modern: `@if`, `@for`; hoofdstuk 2)
- bestandsnamen als `app.component.ts` (modern: `app.ts`, zonder de suffix)
- `standalone: true` in elke component (sinds v19 de standaard, dus ruis)
- uitleg die over `zone.js` begint alsof het de kern van Angular is

Voor deze fase van het vak: gebruik AI gerust om *concepten* uit dit hoofdstuk anders uitgelegd te krijgen ("leg inversion of control uit met een ander voorbeeld", "wat is het verschil tussen dependencies en devDependencies") en om *foutmeldingen* te ontcijferen. Hou angular.dev erbij om te controleren wat het model beweert. Laat AI in deze fase geen projectcode voor je schrijven. Je hebt het referentiekader nog niet om te zien wat eraan scheelt, en dit hoofdstuk toont dat er geregeld iets aan scheelt. Dat referentiekader bouw je de komende hoofdstukken op; de oefeningen met AI groeien mee.

Oefeningen

Drills (zonder AI aanbevolen)

D1.1 Richt je werkplaats in: Node (LTS), npm, Angular CLI, VS Code met Angular Language Service. check de uitvoer van `node -v` en `ng version`.

D1.2 Genereer een project `eerste-app`, start het, en pas `src/app/app.html` aan tot de startpagina jouw naam en de titel van dit vak toont. Maak bewust een typfout in een tag, bekijk wat de terminal en de browser doen, en herstel.

D1.3 Open `package.json` van je project en beantwoord schriftelijk: welke Angular-versie gebruikt je project precies (let op het dakje)? Waarom staat `@angular/cli` niet bij `dependencies`? Welk commando voert `npm start` uit?

D1.4 Verwijder de map `node_modules` van je project en maak het project weer werkend. Welk commando gebruikte je en hoe npm wist wat het moest installeren.

D1.5 Herbouw de teller uit sectie 1.4 twee keer zonder framework-tooling: één keer met je eigen `signal.js`, één keer met Alpine.js via de CDN-tag uit sectie 1.5 (`x-data`, `@click`, `x-text`). Beschrijf in drie zinnen wat beide versies gemeen hebben en waar ze verschillen.

D1.6 Voer `ng g c proef --dry-run` uit. Welke bestanden zou de CLI aanmaken, en waar? Genereer de component daarna echt en zet ze op het scherm met de twee stappen uit sectie 1.10.

Debug en review (zonder AI)

R1.1 Hieronder vier meldingen uit terminals van week 1. Benoem voor elk de vermoedelijke oorzaak en de eerste stap naar een oplossing, zonder ze uit te voeren: (a) `ng: command not found`; (b) `This command is not available when running the Angular CLI outside a workspace`; (c) `Cannot find module '@angular/core'`; (d) `Port 4200 is already in use`.

R1.2 Een medestudent stuurt je een projectzip van 300 MB "omdat het anders niet werkt bij jou". Leg uit wat er misging, wat de juiste manier van delen is, en welk commando de ontvanger nodig heeft.

R1.3 Een webshop verwacht op Black Friday tien keer zoveel bezoekers als normaal. Leg met sectie 1.2 uit waarom dat voor een SPA met een API een kleiner probleem is dan voor een server-side applicatie, en welk deel van de SPA-architectuur die dag toch onder druk komt.

Bouwen en verantwoorden (met AI)

B1.1 Vraag een AI-model naar "een eenvoudige Angular-tutorial voor beginners". Neem de output niet over in een project, maar beoordeel ze. Markeer elk element dat gedateerd is volgens dit hoofdstuk en het lijstje hierboven. Zet er telkens de moderne tegenhanger naast (zoek die op angular.dev als je hem nog niet kent). Sluit af met een oordeel van drie zinnen: houdt een beginner die deze tutorial volgt vandaag een werkend, modern project over?

Verder lezen

- angular.dev - Overview - het officiële startpunt, en jouw referentie voor alles

- angular.dev - Installation - de actuele installatie-instructies
- angular.dev - CLI Reference - elk commando en elke optie van `ng`
- The TypeScript Handbook - voor wie na sectie 1.8 meer wil
- Node.js - About - wat Node is, van de bron zelf